Philosophy of GSP_API Examples

1. Preamble

The `examples/` directory contains 50+ scripts. They are not a random grab-bag — every example follows the same skeleton, and that skeleton encodes the design of the library. Read three examples and you have read all of them; the differences are local to one or two steps. This document names the patterns, explains *why* the API is shaped this way, and gives you a mental model strong enough to write a new example from scratch.

Audience. Someone reading examples to learn GSP_API, or writing a new one to demonstrate a feature.

How to use this document. Each section names a pattern, shows the canonical snippet, and points to the example files that demonstrate it. To verify a claim, open the cited file. To run a pattern under both backends:

```
GSP_RENDERER=matplotlib python examples/<file>.py
GSP_RENDERER=datoviz    python examples/<file>.py
```

> ***Heads-up:*** ***`examples/README.md`*** ***says*** ***`GSP_BACKEND`*** ***, but the code reads*** ***`GSP_RENDERER`*** ***.*** *The factory in examples/common/example_helper.py:42 is the source of truth —* *`os.environ.get("GSP_RENDERER", ...)`* *. Use* *`GSP_RENDERER`* *when running examples; the README is stale.*

---

2. The Five Design Principles

The "why" before the "what".

2.1 Backend independence

The same example script runs unchanged under three backends — `matplotlib` (static, ubiquitous), `datoviz` (GPU, interactive), and `network` (remote rendering over HTTP). A single environment variable picks the implementation:

```
GSP_RENDERER=datoviz python examples/points_example.py
```

Payoff. Examples are simultaneously a tutorial *and* a cross-backend conformance test suite. If a feature renders differently under matplotlib vs datoviz, that is a bug, and the existing example is the regression test.

2.2 Data first, render last

Buffers, visuals, viewports, model matrices, and cameras are all **plain data**. Nothing renders until the explicit terminal call `renderer.render(...)`. There is no implicit drawing, no lazy auto-render on attribute set, no global "current scene".

Payoff. A scene is fully inspectable, serializable, and renderer-swappable up to the moment of rendering. The same in-memory scene can be rendered to PNG, SVG, an interactive window, or a remote server with no changes to the construction code.

2.3 Parallel-list rendering API

The renderer takes four parallel lists, one entry per item to draw:

```
renderer.render(
    [viewport_1, viewport_2, viewport_3, viewport_4],
    [pixels_1,   pixels_2,   pixels_2,   pixels_1],
    [model_matrix, model_matrix, model_matrix, model_matrix],
    [camera, camera, camera, camera],
)
```

Payoff. Composing N panels uses the same code as composing one — extend the lists. There is no scene-graph mutation API to learn; the data structure is the API.

2.4 Typed GPU buffers, not opaque arrays

Visuals do not consume numpy arrays directly. They consume `Buffer` / `Bufferx` objects with an explicit `BufferType` (`vec3` , `rgba8` , `float32` , `mat4` , ...). The standard ramp from numpy to GPU is a one-liner:

```
positions_buffer = Bufferx.from_numpy(positions_numpy, BufferType.vec3)
```

Payoff. Shape and dtype mismatches surface at buffer construction with a clear error, not deep inside a renderer where the stack trace points at GLSL. Buffers are also the unit of GPU residency, so the typed handle makes it explicit when data crosses the CPU/GPU boundary.

2.5 Save + show, always both

Every basic example ends with the same two-line conclusion:

```
ExampleHelper.save_output_image(rendered_image, f"
{pathlib.Path(__file__).stem}_{renderer_name}.png")
renderer_base.show()
```

Payoff. The same script works in CI (PNG lands in `examples/output/` , no display required) and at a developer's desk (interactive window opens). One code path, two audiences.

---

3. The Canonical Skeleton

Every basic example follows the same seven steps. `examples/points_example.py` is the reference template — read it once and the rest of the directory becomes structurally familiar.

Step 1 — Imports

Three tiers, in this order:

```
# stdlib imports
import pathlib

# pip imports
import numpy as np
```

```
# local imports
from common.example_helper import ExampleHelper
from gsp.constants import Constants
from gsp.core import Canvas, Viewport, Camera
from gsp.visuals import Points
from gsp.types import Buffer, BufferType
from gsp_extra.bufferx import Bufferx
from gsp.utils.unit_utils import UnitUtils
```

Stdlib first, then pip dependencies, then GSP modules grouped by package (`common` , `gsp.core` , `gsp.visuals` , `gsp.types` , `gsp_extra` , `gsp.utils`). Comments mark each tier.

Step 2 — Canvas

Pixel-sized, DPI-aware, with an explicit background colour:

```
canvas = Canvas(width=256, height=256, dpi=127.5,
background_color=Constants.Color.white)
```

DPI is load-bearing: it converts between pixels and points (e.g. for line widths via `UnitUtils.pixel_to_point`). Do not omit it.

Step 3 — Viewport

A rectangular region inside the canvas, in pixel coordinates `(x, y, width, height)` :

```
viewport = Viewport(0, 0, canvas.get_width(), canvas.get_height(),
Constants.Color.transparent)
```

Multiple viewports compose by tiling, overlapping, or stacking — see §4.2.

Step 4 — Data preparation

Three idiomatic ways to build a buffer, all visible in `points_example.py` :

```
# (a) From a numpy array
positions_numpy = np.random.rand(point_count, 3).astype(np.float32) * 2.0 - 1
positions_buffer = Bufferx.from_numpy(positions_numpy, BufferType.vec3)

# (b) Constant fill via raw bytes
face_colors_buffer = Buffer(point_count, BufferType.rgba8)
face_colors_buffer.set_data(bytearray([255, 0, 0, 255]) * point_count, 0,
point_count)

# (c) DPI-aware scalar via a unit utility
edge_widths_numpy = np.array([UnitUtils.pixel_to_point(1, canvas.get_dpi())] *
point_count, dtype=np.float32)
edge_widths_buffer = Bufferx.from_numpy(edge_widths_numpy, BufferType.float32)
```

Step 5 — Visual instantiation

Visuals consume buffers; they are dumb data containers, not draw commands:

```
points = Points(positions_buffer, sizes_buffer, face_colors_buffer,
edge_colors_buffer, edge_widths_buffer)
```

Step 6 — Camera and model matrix

The default identity transform produces an NDC-style view, suitable for 2D layouts:

```
model_matrix = Bufferx.mat4_identity()
camera = Camera(Bufferx.mat4_identity(), Bufferx.mat4_identity())
```

For 3D scenes, the same handles accept perspective/ortho projections and view matrices — see §4.5.

Step 7 — Render, save, show

```
renderer_name  = ExampleHelper.get_renderer_name()
renderer_base  = ExampleHelper.create_renderer(renderer_name, canvas)
rendered_image = renderer_base.render([viewport], [points], [model_matrix],
[camera])

ExampleHelper.save_output_image(rendered_image, f"
{pathlib.Path(__file__).stem}_{renderer_name}.png")
renderer_base.show()
```

The output filename pattern `{stem}_{renderer_name}.png` ensures `matplotlib` and `datoviz` runs of the same example produce side-by-side artefacts in `examples/output/`, which is exactly what you want for visual regression diffs.

---

4. Pattern Catalog

4.1 Buffer construction patterns

Use case	Snippet	Notes
numpy → GPU buffer	`Bufferx.from_numpy(arr, BufferType.vec3)`	Zero-copy where possible; the `BufferType` is required and validates shape/dtype.
Constant fill	`Buffer(n, BufferType.rgba8); buf.set_data(bytes_pattern * n, 0, n)`	`set_data` takes raw bytes — for rgba8, that's 4 bytes per element.
Colormap	`CmapUtils.get_color_map("plasma", normalized_values)`	Returns an rgba8 buffer keyed off a numpy array of normalised floats.
DPI-aware sizes	`UnitUtils.pixel_to_point(1, canvas.get_dpi())`	Use this anywhere a width is in points (line widths, edge widths, font sizes).

Demonstrated in: `points_example.py`, `markers_example.py`, `buffer_example.py`, `image_example.py`.

4.2 Multi-viewport composition

Multiple viewports render in a single call by extending the four parallel lists. Each list position is one *(viewport, visual, model_matrix, camera)* quadruple:

```
renderer_base.render(
    [viewport_1, viewport_2, viewport_3, viewport_4],
    [pixels_1,   pixels_2,   pixels_2,   pixels_1],
    [model_matrix, model_matrix, model_matrix, model_matrix],
    [camera, camera, camera, camera],
)
```

Layouts emerge from how you place the viewports in pixel coordinates:

- **Tiled grid** — `Viewport(0, 0, w/2, h/2)`, `Viewport(w/2, 0, w/2, h/2)`, … as in `viewport_multi_example.py`.
- **Overlapping** — viewports whose rectangles intersect; later list entries paint over earlier ones. See `viewport_overlapping_example.py`.
- **Stacked / event-driven** — vertical stacks with input attached, in `viewport_events_example.py`.

Demonstrated in: `viewport_multi_example.py`, `viewport_overlapping_example.py`, `viewport_events_example.py`.

4.3 Interaction: ViewportEvents + AxesPanZoom

Input is delivered via a backend-agnostic `ViewportEvents` object, manufactured by the helper:

```
viewport_events = ExampleHelper.create_viewport_events(renderer_base, viewport)

viewport_events.mouse_move_event.subscribe(on_mouse_move)
viewport_events.button_press_event.subscribe(on_button_press)
viewport_events.mouse_scroll_event.subscribe(on_mouse_scroll)
```

Higher-level controllers consume those events and translate them into camera/axes changes:

```
axes_pan_zoom = AxesPanZoom(viewport_events, base_scale=1.1,
axes_display=axes_display)
axes_display.new_limits_event.subscribe(re_render_callback)
```

`AxesPanZoom` itself contains no rendering logic — it only mutates `axes_display` limits in response to events. The re-render is triggered by subscribing to `new_limits_event`. This separation (events → controller → display → render trigger) is the canonical interactive shape.

Demonstrated in: `viewport_events_example.py`, `vispy_axes_panzoom_example.py`, `vispy_axes_multiple_panzoom_example.py`, `camera_control_example.py`.

4.4 Axes layers (managed → display → panzoom)

Three abstraction levels, increasing in control:

Layer	Purpose	When to reach for it
`AxesManaged`	Wraps a viewport, auto-handles pan/zoom, labels, and titles.	Tutorial code, quick plots, demos. See `vispy_axes_managed_example.py`.
`AxesDisplay`	Exposes the viewport, transform matrix, and `new_limits_event`. No interaction baked in.	When you want custom event wiring. See `vispy_axes_display_example.py`.
`AxesPanZoom`	Pure input controller — listens to `ViewportEvents`, mutates an `AxesDisplay`.	Pair with `AxesDisplay` for full control. See `vispy_axes_panzoom_example.py`.

The split is deliberate: rendering, display state, and interaction can each be replaced independently.

4.5 3D scenes: manual matrices vs Object3D hierarchies

There are two complementary approaches, both valid.

Manual matrices. For one-off transforms, build matrices directly with `glm`:

```
camera_world = np.eye(4, dtype=np.float32) @ glm.translate(np.array([0.0, 0.0, 2.0],
dtype=np.float32))
projection_matrix_numpy = glm.ortho(-1.0, 1.0, -1.0, 1.0, 0.1, 100.0)
projection_matrix_buffer = Bufferx.from_numpy(np.array([projection_matrix_numpy],
dtype=np.float32), BufferType.mat4)
camera.set_projection_matrix(projection_matrix_buffer)

view_matrix_numpy = np.linalg.inv(camera_world)
view_matrix_buffer = Bufferx.from_numpy(np.array([view_matrix_numpy]),
BufferType.mat4)
camera.set_view_matrix(view_matrix_buffer)
```

Object3D hierarchies. For scenes with parent/child relationships and per-node Euler angles:

```
object3d_scene = Object3D("Main Scene")
object3d_pixel = Object3D("Pixels Object3D")
object3d_pixel.attach_visual(pixels)
object3d_scene.add(object3d_pixel)

object3d_pixel.euler[0] = np.pi / 4
object3d_pixel.euler[1] = np.pi / 4
object3d_pixel.euler[2] = np.pi / 4

viewports, visuals, model_matrices, cameras = Object3D.pre_render(viewport,
object3d_scene, camera)
renderer_base.render(viewports, visuals, model_matrices, cameras)
```

`Object3D.pre_render` is the bridge: it flattens a hierarchy into the four parallel lists the renderer expects. The renderer's API does not change — only the way the lists are produced.

Demonstrated in: `simple_model_matrix.py`, `object3d_example.py`, `camera_control_example.py`, `vispy_basic_example.py`.

4.6 Animation: `@animator.event_listener`

Animation is event-driven, stateless per frame, and minimises GPU work via delta rendering. The pattern:

```
animator = ExampleHelper.create_animator(renderer_base)

@animator.event_listener
def animator_callback(delta_time: float) -> list[VisualBase]:
    sizes_numpy = np.random.rand(point_count).astype(np.float32) * 40.0 + 10.0
    sizes_buffer.set_data(bytearray(sizes_numpy.tobytes()), 0,
sizes_buffer.get_count())
    return [points]  # only the visuals that changed

animator.start([viewport], [points], [model_matrix], [camera])
```

Three rules:

1. **Mutate buffers in place** with `set_data(...)`. Do not rebuild the visual.
2. **Return only the changed visuals** from the callback. The renderer skips work for the rest.
3. **`delta_time` is monotonic seconds** since the last frame — use it for time-based animation rather than reading wall-clock.

Video export uses a separate factory:

```
animator = ExampleHelper.create_animator_with_video(
    renderer_base, video_path, fps=60, video_duration=10.0,
)

@animator.on_video_saved.event_listener
def on_save():
    animator.stop()
    renderer_base.close()
```

Same callback, same `start(...)`; the animator writes an `.mp4` and emits `on_video_saved` when the duration is reached.

Demonstrated in: `animator_example.py`, `texts_animated_example.py`, `dynamic_groups_example.py`, `object3d_example.py`.

4.7 Groups: index → attribute association

Groups let one visual carry heterogeneous attributes by mapping vertex indices to attribute slots:

```
groups = [
    [i for i in range(len(positions_numpy)) if positions_numpy[i][1] >  0],
    [i for i in range(len(positions_numpy)) if positions_numpy[i][1] <= 0],
]
colors_buffer = Buffer(2, BufferType.rgba8)
colors_buffer.set_data(Constants.Color.red + Constants.Color.green, 0, 2)
pixels.set_attributes(colors=colors_buffer, groups=groups)
```

Calling `set_attributes(...)` again with new groups remaps colours without recreating the visual or its position buffer — this is the basis of `dynamic_groups_example.py` 's real-time re-styling. `GroupUtils.get_group_count(...)` (visible in `viewport_multi_example.py` and `object3d_example.py`) computes group counts when each vertex is its own group.

Demonstrated in: `groups_example.py` , `dynamic_groups_example.py` , `viewport_multi_example.py` .

4.8 Sessions: timestamped Pydantic snapshots

A "session" is a sequence of `(timestamp, serialized_scene)` pairs. Recording produces it; playback consumes it.

Recording:

```
pydantic_serializer = PydanticSerializer(canvas)
serialized_data: PydanticDict = pydantic_serializer.serialize(
    viewports=[viewport],
    visuals=[points],
    model_matrices=[model_matrix],
    cameras=[camera],
)
gsp_session.items.append(
    PydanticSessionItem(timestamp=relative_timestamp,
serialized_data=serialized_data),
)
```

The recorder skips redundant frames by comparing `serialized_data` for equality with the previous item — back-to-back identical scenes don't bloat the session.

Playback:

```
pydantic_parser = PydanticParser()
_, viewports, visuals, model_matrices, cameras =
pydantic_parser.parse(session_item.serialized_data)
renderer_base.render(viewports, visuals, model_matrices, cameras)
```

A session can be persisted to JSON, shipped over the wire, or replayed in CI for regression testing.

Demonstrated in: `session_record_example.py` , `session_player_example.py` .

4.9 Transforms and serialization

Two complementary serialization paths exist.

`TransformChain` (data pipelines). A chain of operations on a buffer, serializable to JSON, deserializable back to an executable chain:

```
transform_serialized = transform_chain.serialize()
# … later, possibly elsewhere …
transform_deserialized = TransformChain.deserialize(transform_serialized)
```

This is the right tool when the pipeline itself is what you want to ship — for lazy evaluation, remote execution, or recording the construction recipe rather than the final array.

Pydantic full-scene serialization. As in §4.8, but used standalone in `pydantic_cycle_example.py` to demonstrate full round-trip of a scene (visuals + buffers + transforms) through Pydantic dictionaries.

Demonstrated in: `transform_example.py`, `transform_build_sample.py`, `transform_serialization_example.py`, `transform_visual_example.py`, `pydantic_cycle_example.py`.

4.10 Vector and remote output

Vector formats. The matplotlib renderer accepts an `image_format` argument; the same scene comes back as PNG, SVG, or PDF bytes:

```
rendered_png = renderer.render([viewport], [paths], [model_matrix], [camera],
image_format="png")
rendered_svg = renderer.render([viewport], [paths], [model_matrix], [camera],
image_format="svg")
rendered_pdf = renderer.render([viewport], [paths], [model_matrix], [camera],
image_format="pdf")
```

See `svg_pdf_example.py`.

Remote rendering. `NetworkRenderer` implements the same `RendererBase` contract but sends the scene over HTTP to a remote `gsp_server` (which itself runs matplotlib or datoviz):

```
renderer = NetworkRenderer(canvas, server_base_url="http://localhost:5000",
remote_renderer_name="datoviz")
rendered_image = renderer.render([viewport], [pixels], [model_matrix], [camera])
```

Local code is identical to single-machine code. The choice of remote backend is parameterised — set `GSP_REMOTE_RENDERER=matplotlib|datoviz` when `GSP_RENDERER=network`.

Demonstrated in: `svg_pdf_example.py`, `network_client_example.py`.

---

5. The `common/` Helpers Reference

Three files live in examples/common/:

`ExampleHelper` (example_helper.py)

The factory that hides backend selection. Most examples use it; the few that instantiate `MatplotlibRenderer` / `DatovizRenderer` / `NetworkRenderer` directly do so deliberately to demonstrate a backend-specific feature.

Method	What it does
`get_renderer_name()`	Reads `GSP_RENDERER` env var, defaults to `matplotlib`. Asserts the value is one of `matplotlib

`get_remote_renderer_name()`	For the `network` renderer only — reads `GSP_REMOTE_RENDERER`, defaults to `matplotlib`.
`create_renderer(name, canvas)`	Returns `MatplotlibRenderer
`create_animator(renderer)`	Returns the animator paired with the given renderer.
`create_animator_with_video(renderer, path, fps, duration)`	Same, but configured to record to an MP4.
`create_viewport_events(renderer, viewport)`	Returns the `ViewportEvents` implementation matching the renderer.
`save_output_image(bytes, basename)`	Writes to `examples/output/<basename>` and prints the resolved path.

The default renderer is set on a class attribute (`ExampleHelper.default_renderer_name`), so flipping the in-source default for local experiments takes one edit.

`Bufferx` (in `gsp_extra.bufferx`)

Helpers around `Buffer` :

- `Bufferx.from_numpy(array, type)` — the standard numpy → GPU bridge.
- `Bufferx.to_numpy(buffer)` — GPU → numpy round-trip for inspection or assertions.
- `Bufferx.mat4_identity()` — the identity model/view matrix used by every "no transform" case.

Other utilities

- `CmapUtils.get_color_map(name, normalized_values)` — colormap sampling into an rgba8 buffer.
- `TextureUtils.*` — image loading helpers for textured visuals.
- `UnitUtils.pixel_to_point(pixels, dpi)` — DPI-aware unit conversion (used for line widths and font-like sizes).
- `GroupUtils.get_group_count(vertex_count, groups)` — computes the right group-buffer length when groups are uniform.

`asset_downloader.py` , `big_tester_helper.py`

Narrower helpers. `asset_downloader` fetches sample data files referenced by examples; `big_tester_helper` is the workhorse for the stress-test scripts (`_big_tester_*`). Read them only when working on those specific examples.

---

6. Writing a New Example: A Checklist

The "now you do it" section. Each box mirrors a step in the canonical skeleton or a project convention.

- [ ] **One feature per file.** Name it `<feature>_example.py` (e.g. `polylines_example.py`).
- [ ] **Module docstring** on line 1 explaining what the example demonstrates.
- [ ] **Three-tier imports** with comments: `# stdlib imports` , `# pip imports` , `# local imports` .
- [ ] **`main()` function** containing all logic; `if __name__ == "__main__": main()` guard at the bottom.

- [ ] **Reproducible randomness.** `np.random.seed(0)` at the top of `main()` if you generate data — keeps output diffs sane.
- [ ] **Prefer `ExampleHelper.create_renderer`.** Instantiate `MatplotlibRenderer` / `DatovizRenderer` / `NetworkRenderer` directly only when the example is demonstrating a backend-specific feature (as `svg_pdf_example.py`, `network_client_example.py`, `viewport_inch_matplotlib.py`, and `dynamic_groups_example.py` do).
- [ ] **Both save and show.** End with `ExampleHelper.save_output_image(rendered_image, f"{pathlib.Path(__file__).stem}_{renderer_name}.png")` followed by `renderer_base.show()`.
- [ ] **Verify under both backends:**

  ```
  GSP_RENDERER=matplotlib python examples/<your_example>.py
  GSP_RENDERER=datoviz    python examples/<your_example>.py
  ```

  Look for both PNGs in `examples/output/`. They should be visually equivalent.
- [ ] **For animated examples**, use `@animator.event_listener` and return only changed visuals.
- [ ] **For interactive examples**, build the chain `ExampleHelper.create_viewport_events → AxesPanZoom → AxesDisplay.new_limits_event → re-render` rather than wiring callbacks ad hoc.
- [ ] **Add a row** to the appropriate table in `examples/README.md`.

---

7. Naming and Filename Conventions

Convention	Meaning
`<feature>_example.py`	Public, supported example. Belongs in `examples/README.md`.
`_<name>.py` (underscore prefix)	Experimental, internal, or work-in-progress. Not shipped as documentation. Examples: `_axes_image_pyramid_*.py`, `_mesh_dvz_manual.py`, `_big_tester_*.py`.
`vispy_*.py`	Examples that exercise the vispy-style axes / pan-zoom stack specifically. (Despite the name, they still run under matplotlib and datoviz; the `vispy_` prefix is historical.)
`examples/output/<stem>_<renderer>.png`	Rendered output destination. The renderer suffix lets matplotlib and datoviz outputs coexist.
`examples/data/`, `examples/models/`, `examples/images/`	Sample assets used by examples.
`examples/expected/`	Reference output snapshots for regression comparison.

---

8. Verification: how to read this document against the code

Every claim in this document is grounded in a specific file. To verify:

1. **For a pattern in §4** — open the cited example file, locate the snippet quoted here, and run it under both backends as shown in §1. The output PNGs in `examples/output/` are the proof.

2. **For an `ExampleHelper` method** — see examples/common/example_helper.py; each method is small enough to read in full.
3. **For the env-var name** — `grep -rn "GSP_RENDERER\|GSP_BACKEND" examples/ src/` . Only example_helper.py:42 uses `GSP_RENDERER` from code; `GSP_BACKEND` appears only in the (stale) README.
4. **For a snippet quoted verbatim** — `grep -n "<snippet>" examples/<file>.py` . Examples cited above:
   - `grep -n "ExampleHelper.create_renderer" examples/*.py`
   - `grep -n "@animator.event_listener" examples/animator_example.py`
   - `grep -n "Object3D.pre_render" examples/object3d_example.py`

When the code drifts from this document — change the document. The examples are the source of truth; this file just names what they already do.